🎓 Модуль 3: Ансамблевые методы в машинном обучении

ЧАСТЬ 1. ВВЕДЕНИЕ В АНСАМБЛЕВЫЕ МЕТОДЫ

1.1. Что такое ансамбль?

Ансамбль (от французского *ensemble* — «вместе, совокупность») в машинном обучении — это набор моделей, объединённых для получения более качественного предсказания, чем каждая модель по отдельности.

Простыми словами: вместо одной умной модели мы обучаем **много «слабых» моделей** и объединяем их мнения. В большинстве случаев такой «коллектив» работает лучше, чем любой отдельный эксперт.

Формальное определение

Пусть даны:

- **Обучающая выборка:** $D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$
- **Базовые модели (base learners):** $h_1, h_2, \ldots, h_M$
- **Мета-алгоритм объединения:** $F(h_1, h_2, \ldots, h_M)$

Тогда **ансамбль** — это композиция: $H(x) = F(h_1(x), h_2(x), \ldots, h_M(x))$

Историческая справка

Год	Событие
1990-е	Появление первых ансамблевых методов (Bagging, Boosting)
1995	Лео Брейман предлагает **Bagging**

1996	Фрейнд и Шапире разрабатывают **AdaBoost**
2001	Брейман публикует **Random Forest** — самый популярный ансамбль
2014	Тянци Чен создаёт **XGBoost** — революция в соревнованиях Kaggle
2017	Microsoft выпускает **LightGBM**
2017	Яндекс публикует **CatBoost**
2018+	**Optuna**, **SHAP** — стандарты тюнинга и интерпретации

1.2. Философия «мудрости толпы»

Аналогия из жизни

Представьте, что вы хотите оценить вес быка на ярмарке. Есть два подхода:

Подход 1: один эксперт Спросите одного опытного фермера. Он даст оценку, например, 900 кг. Но может ошибиться на 100 кг.

Подход 2: множество экспертов Спросите 100 случайных людей (не экспертов) — туристов, детей, стариков. Каждый даст очень грубую оценку: кто-то скажет 500 кг, кто-то — 1500 кг. Но если **усреднить** все 100 оценок — получится очень точный результат!

Именно так работает ансамбль: **слабые модели, объединённые правильно, образуют сильную систему**.

Почему это работает?

Это явление называется **«мудрость толпы»** (Wisdom of the Crowd). Фундамент — **теорема Кондорсе о жюри присяжных** (1785):

> *Если каждый член жюри принимает правильное решение с вероятностью $p > 0.5$, то вероятность правильного решения большинства стремится к 1 при росте числа членов жюри.*

1.3. Зачем нужны ансамблевые методы?

Проблемы одиночных моделей

Проблема	Описание	Пример
Переобучение	Модель запоминает шум	Одиночное решающее дерево
Высокая дисперсия	Модель слишком чувствительна к данным	KNN, Decision Tree
Высокое смещение	Модель слишком простая	Линейная регрессия
Локальные минимумы	Модель «застревает» в неоптимальном решении	Нейросети

Что решают ансамбли

Ансамбли позволяют **снизить дисперсию** (bagging) или **снизить смещение** (boosting), не теряя при этом обобщающую способность.

Компромисс смещения и дисперсии (Bias-Variance Tradeoff)

Ошибка модели раскладывается на три компоненты:

$$Error = Bias^2 + Variance + Noise$$

Компонента	Что означает	Как борется ансамбль
Bias (смещение)	Насколько модель «промахивается» в среднем	**Boosting** снижает bias
Variance (дисперсия)	Насколько предсказания «разбросаны»	**Bagging** снижает variance

Noise (шум)	Неустранимая случайность	Не устраняется

ЧАСТЬ 2. КЛАССИФИКАЦИЯ АНСАМБЛЕВЫХ МЕТОДОВ

2.1. Три главные стратегии

Все ансамблевые методы делятся на три большие группы по способу обучения базовых моделей:

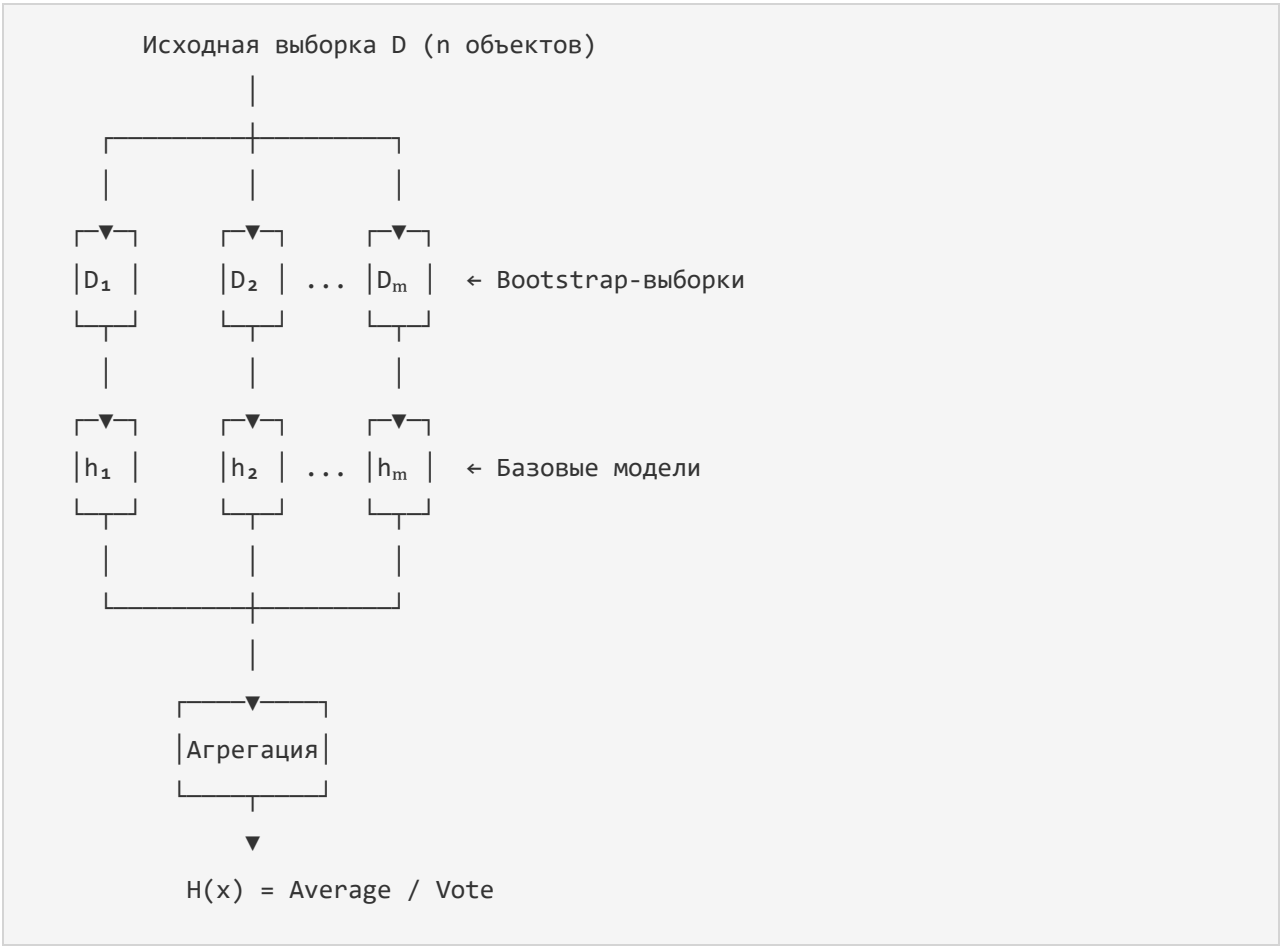
Стратегия	Как обучаются модели	Цель	Примеры
Bagging	Параллельно на разных подвыборках	Снизить **variance**	Random Forest
Boosting	Последовательно, исправляя ошибки	Снизить **bias**	XGBoost, LightGBM, CatBoost
Stacking	Параллельно + мета-модель	Комбинация сильных моделей	StackingClassifier

2.2. Bagging (Bootstrap Aggregating)

Идея

1. Из обучающей выборки **случайно с возвращением** берётся подвыборка того же размера
2. На этой подвыборке обучается базовая модель
3. Шаги 1-2 повторяются M раз — получаем M моделей
4. Итоговое предсказание = **усреднение** (для регрессии) или **голосование** (для классификации)

Схема



Особенности Bagging

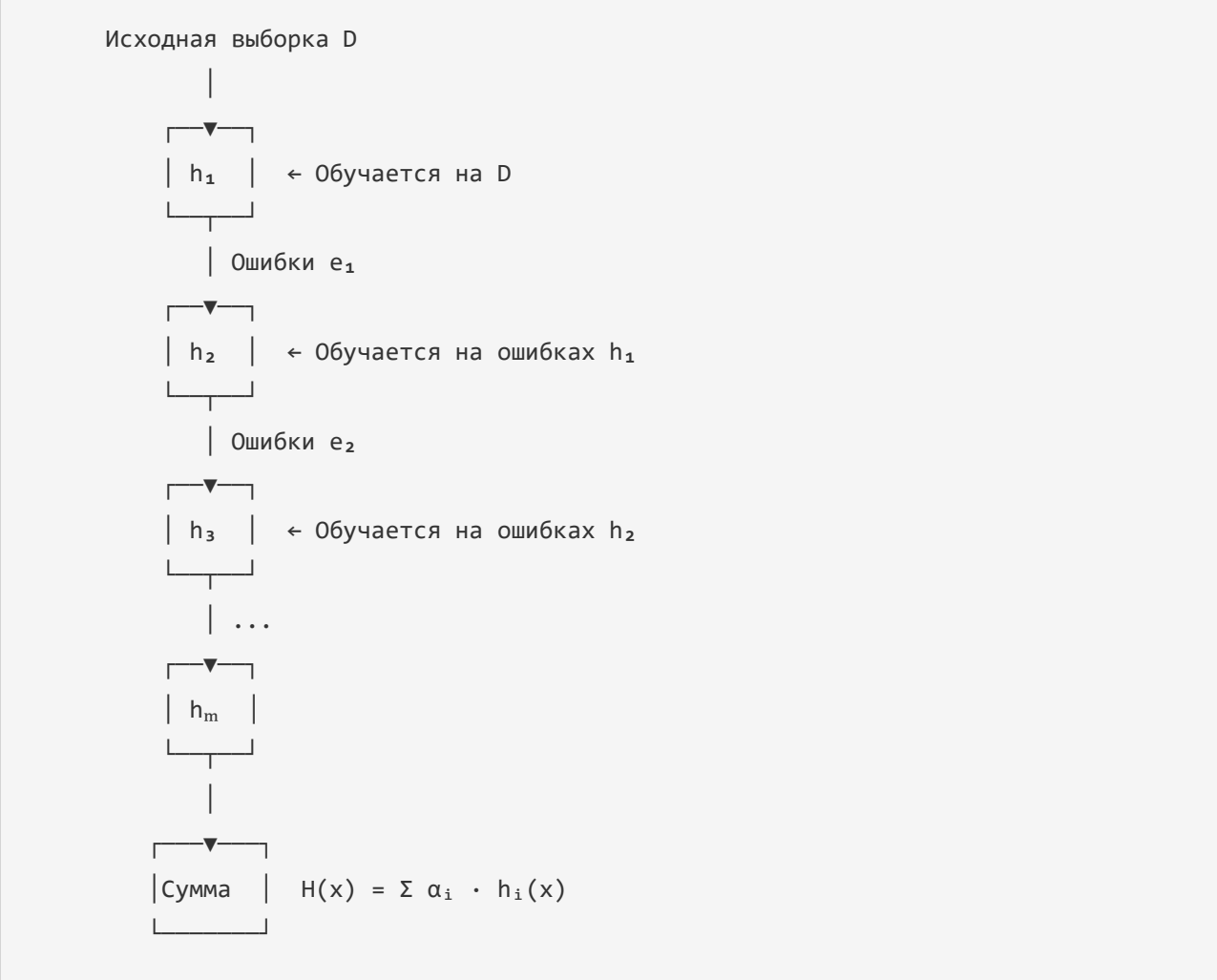
- **Уменьшает дисперсию**, не меняя смещение
- Хорошо работает со **«слабыми» моделями с высокой дисперсией** (глубокие деревья)
- Плохо работает с **устойчивыми моделями** (линейная регрессия)
- Легко **параллелизуется**

2.3. Boosting

Идея

1. Обучается первая модель (обычно простая)
2. Анализируются объекты, где модель ошиблась
3. Обучается вторая модель, **фокусируясь на ошибках** первой
4. Процесс повторяется — каждая новая модель исправляет ошибки предыдущих
5. Итоговое предсказание = **взвешенная сумма** всех моделей

Схема



Особенности Boosting

- **Уменьшает смещение**, может незначительно увеличить дисперсию
- Работает с **«слабыми» моделями с высоким смещением** (неглубокие деревья, «пни»)
- **Чувствителен к шуму и выбросам**
- Требует **последовательного обучения** (сложнее параллелить)

Типы Boosting

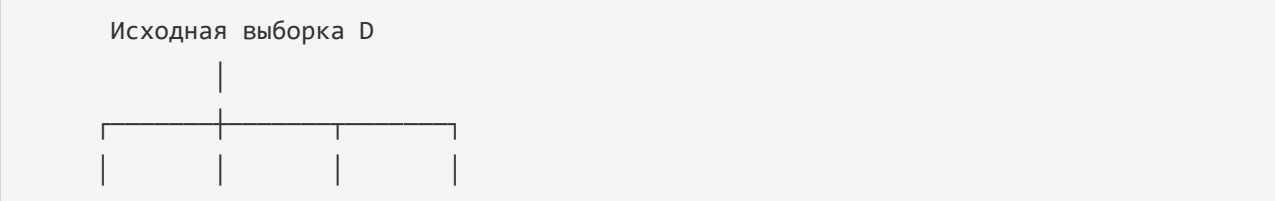
Тип	Как работает	Пример
AdaBoost	Увеличивает веса ошибочных объектов	AdaBoost
Gradient Boosting	Каждая модель предсказывает **градиент ошибки**	XGBoost, LightGBM
CatBoost	Упорядоченный boosting + авто-категории	CatBoost

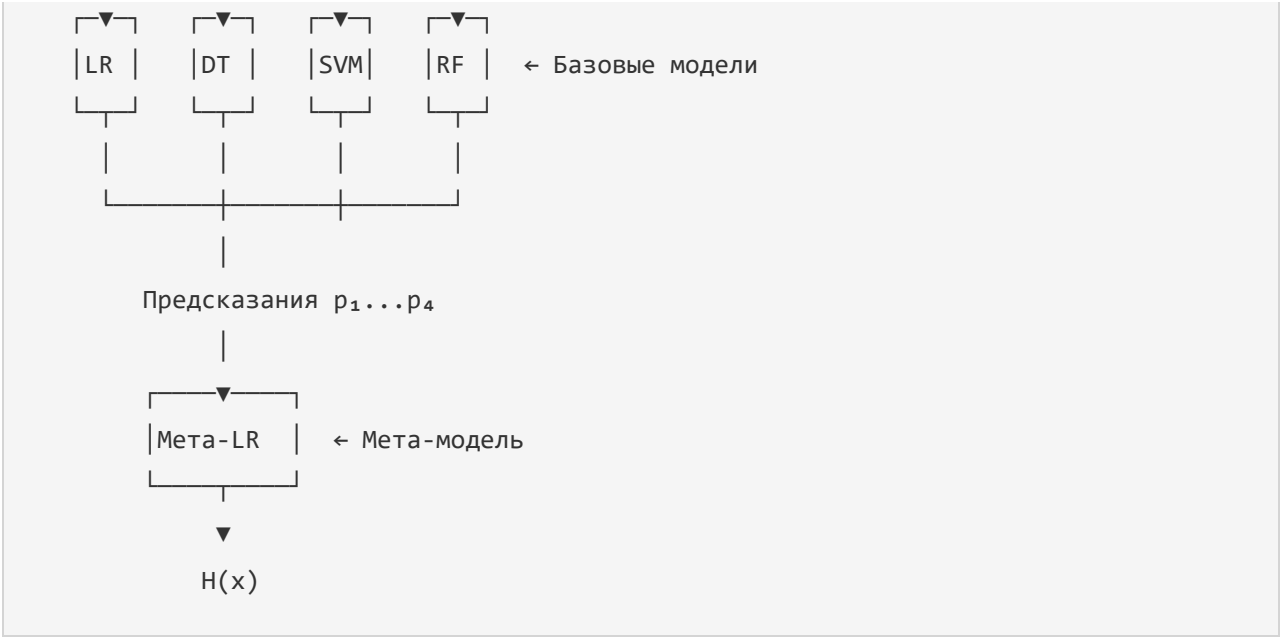
2.4. Stacking

Идея

1. Обучается несколько **разнородных** базовых моделей (LR, DT, SVM...)
2. Их предсказания подаются на вход **мета-модели**
3. Мета-модель учится комбинировать предсказания базовых

Схема





Особенности Stacking

- Работает с **разнородными** моделями
- Мета-модель учится **оптимально взвешивать** базовые
- Требует **кросс-валидации** для избежания утечки
- Может дать **наибольший прирост** качества

2.5. Сравнительная таблица

Характеристика	Bagging	Boosting	Stacking
Обучение	Параллельно	Последовательно	Параллельно + мета
Базовые модели	Однородные	Однородные	Разнородные
Что снижает	Variance	Bias	И то, и другое
Скорость	Быстро	Медленно	Средне
Переобучение	Устойчиво	Чувствительно	Устойчиво

Пример	Random Forest	XGBoost	StackingClassifier

ЧАСТЬ 3. ОСНОВНЫЕ АЛГОРИТМЫ

3.1. Random Forest

Тип: Bagging + Decision Trees

Автор: Лео Брейман (2001)

Сложность обучения: $O(M \cdot nlog\ n \cdot \sqrt{p})$

Алгоритм

1. Для каждого из M деревьев:
   - Взять **bootstrap-выборку** (n объектов с возвращением)
   - Построить дерево решений, но на каждом разбиении рассматривать **случайное подмножество** из $\sqrt{p}$ признаков
2. Итоговое предсказание = **голосование** (классификация) или **среднее** (регрессия)

Ключевые особенности

- **Декорареляция деревьев** за счёт случайного выбора признаков
- **OOB-оценка** (Out-of-Bag) — бесплатная валидация
- **Важность признаков** — встроенная интерпретация
- **Устойчивость к выбросам** и шуму

Гиперпараметры

Параметр	Значение по умолчанию	Что влияет
`n_estimators`	100	Число деревьев
`max_depth`	None	Глубина (None = до листьев)
`max_features`	`'sqrt'`	Признаков на разбиение
`min_samples_split`	2	Минимум образцов для разбиения
`min_samples_leaf`	1	Минимум образцов в листе
`bootstrap`	True	Bootstrap-выборка
`class_weight`	None	Веса классов

3.2. XGBoost

Тип: Gradient Boosting
Автор: Тянци Чен (2014)
Основной инструмент победителей Kaggle

Ключевые особенности

1. **Регуляризация:** штраф за сложность дерева \mathcal{L} = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k) где $\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_j w_j^2$
2. **Приближённое решение:** использует второй порядок Тейлора для аппроксимации
3. **Sparsity-aware:** обрабатывает пропуски автоматически
4. **Column Block:** параллельное обучение по признакам
5. **Cache-aware:** оптимизация работы с памятью

Гиперпараметры

Параметр	Типичное значение	Влияние
`n_estimators`	100-1000	Число итераций
`learning_rate`	0.01-0.3	Скорость обучения
`max_depth`	3-10	Глубина дерева
`subsample`	0.5-1.0	Доля объектов
`colsample_bytree`	0.5-1.0	Доля признаков
`reg_alpha`	0-1	L1-регуляризация
`reg_lambda`	1-10	L2-регуляризация
`gamma`	0-5	Мин. снижение для разбиения

3.3. LightGBM

Тип: Gradient Boosting
Автор: Microsoft (2017)
Быстрее XGBoost в 10-20 раз

Ключевые особенности

1. **Leaf-wise рост дерева** вместо level-wise:
   - XGBoost: растёт уровень за уровнем
   - LightGBM: растёт самый «перспективный» лист
   - Результат: меньше деревьев, выше точность
2. **GOSS** (Gradient-based One-Side Sampling): отбрасывает объекты с малым градиентом
3. **EFB** (Exclusive Feature Bundling): объединяет разреженные признаки

4. **Гистограммный алгоритм:** биннинг признаков

Гиперпараметры

Параметр	Типичное значение	Влияние
`n_estimators`	100-1000	Число итераций
`num_leaves`	20-150	Число листьев (важнее max_depth!)
`max_depth`	3-12	Глубина
`learning_rate`	0.01-0.3	Скорость обучения
`min_child_samples`	5-50	Минимум в листе
`subsample`	0.5-1.0	Доля объектов
`colsample_bytree`	0.5-1.0	Доля признаков

3.4. CatBoost

Тип: Gradient Boosting
Автор: Яндекс (2017)
Лучший для данных с категориями

Ключевые особенности

1. **Автоматическая обработка категорий:**
   - Не требует One-Hot Encoding
   - Использует **упорядоченное целевое кодирование** (ordered target encoding)
2. **Упорядоченный boosting:**
   - Снижает **смещение** (bias) в оценке градиента

- Помогает бороться с переобучением

3. **Симметричные деревья:**
   - Все узлы на одном уровне используют **одинаковое** разбиение
   - Быстрый инференс
4. **GPU-обучение** из коробки

Гиперпараметры

Параметр	Типичное значение	Влияние
`iterations`	100-1000	Число итераций
`depth`	3-10	Глубина
`learning_rate`	0.01-0.3	Скорость обучения
`l2_leaf_reg`	1-10	L2-регуляризация
`border_count`	32-255	Корзин для числовых
`cat_features`	—	Индексы категорий

3.5. Сравнение алгоритмов

Критерий	Random Forest	XGBoost	LightGBM	CatBoost
Тип	Bagging	Boosting	Boosting	Boosting
Скорость обучения	Средняя	Средняя	**Быстрая**	Медленная

Скорость инференса	Средняя	Средняя	Быстрая	**Быстрая**
Категории	Нужен encoding	Нужен encoding	Нужен encoding	**Автоматически**
Пропуски	Через imputation	Авто	Авто	Авто
Переобучение	Устойчив	Средне	Средне	Устойчив
Тюнинг	Простой	Средний	Сложный	Простой
Когда использовать	Baseline	Универсал	Большие данные	Много категорий

ЧАСТЬ 4. ГДЕ ПРИМЕНЯЮТСЯ АНСАМБЛИ

4.1. Финансы и банки

Задача	Алгоритм	Почему ансамбль
Кредитный скоринг	XGBoost, LightGBM	Много признаков, дисбаланс классов
Обнаружение мошенничества	Random Forest, CatBoost	Аномалии, редкие события
Оценка рисков	Gradient Boosting	Стоимость ошибки высока
Алготрейдинг	Random Forest	Нелинейные зависимости

4.2. Электронная коммерция

Задача	Алгоритм	Почему ансамбль
Рекомендации товаров	LightGBM	Большие объёмы, скорость
Прогноз спроса	XGBoost	Сезонность, много факторов
Отток клиентов	CatBoost	Категориальные признаки
Динамическое ценообразование	Gradient Boosting	Нелинейные зависимости

4.3. Медицина и биология

Задача	Алгоритм	Почему ансамбль
Диагностика заболеваний	Random Forest, XGBoost	Ответственность, интерпретируемость
Прогноз госпитализации	Gradient Boosting	Много факторов риска
Анализ генома	Random Forest	Тысячи признаков
Прогноз выживаемости	XGBoost	Учёт цензурирования

4.4. Промышленность и нефтегаз

Задача	Алгоритм	Почему ансамбль
Прогноз отказов оборудования	LightGBM	Датчики, временные ряды
Дебит скважин	XGBoost	Много геологических признаков

Контроль качества	Random Forest	Много сенсоров
Оптимизация бурения	CatBoost	Категориальные геоданные

4.5. IT и телеком

Задача	Алгоритм	Почему ансамбль
Классификация тикетов	LightGBM + TF-IDF	Тексты + категории
Прогноз нагрузки	Gradient Boosting	Временные ряды
Churn клиентов	CatBoost	Категории (тариф, регион)
Ранжирование поиска	LambdaMART (GB)	Специальная функция потерь

4.6. Соревнования по ML

На **Kaggle** практически все топовые решения используют ансамбли:

- **Первое место** обычно: **Stacking** из XGBoost + LightGBM + CatBoost + нейросети
- **Популярные приёмы:** blending, bagging, seed averaging
- **Известные кейсы:**
  - Otto Group Product Classification (2015) — стек из 30+ моделей
  - Home Credit Default Risk (2018) — XGBoost + LightGBM + CatBoost

---

ЧАСТЬ 5. ПРЕИМУЩЕСТВА И НЕДОСТАТКИ

5.1. Преимущества ансамблей

Преимущество	Описание
Высокая точность	Превосходят одиночные модели
Устойчивость к переобучению	Особенно bagging
Работа с дисбалансом	Через `class_weight`, scale_pos_weight
Универсальность	Работают и с табличными, и с текстовыми данными
Автоматический отбор признаков	Встроенная важность
Обработка пропусков	XGBoost, LightGBM, CatBoost
Масштабируемость	Хорошо работают на больших данных

5.2. Недостатки ансамблей

Недостаток	Описание
Сложность интерпретации	«Чёрный ящик» (решается через SHAP)
Вычислительная сложность	Особенно boosting
Память	Много деревьев = много памяти
Требует тюнинга	Гиперпараметров много
Не всегда лучше одиночных	На малых данных может переобучаться

5.3. Когда ансамбли НЕ нужны

- **Мало данных** (< 1000 объектов) — лучше линейные модели

- **Простая задача** — одиночное дерево или логистическая регрессия справится
- **Нужна максимальная интерпретируемость** — используйте линейные модели
- **Real-time инференс с жёстким ограничением** — рассмотрите одиночное дерево

ЧАСТЬ 6. ТЮНИНГ ГИПЕРПАРАМЕТРОВ

6.1. Почему это важно?

Гиперпараметры **определяют поведение модели**. Неправильный выбор — катастрофическое качество. Правильный выбор — рост на 3-10% метрики.

6.2. Три подхода к поиску

Метод	Как работает	Плюсы	Минусы
GridSearchCV	Перебор **всех** комбинаций	Надёжно	Очень медленно
RandomizedSearchCV	Случайные N комбинаций	Быстро	Может пропустить оптимум
Optuna	Байесовская оптимизация	Самый эффективный	Требует установки

6.3. Optuna — современный стандарт

Optuna использует **TPE** (Tree-structured Parzen Estimator) — байесовский подход, который после каждого испытания строит вероятностную модель и предлагает следующее значение там, где ожидается улучшение.

Основные компоненты

- **Objective function** — целевая функция (что оптимизируем)
- **Study** — процесс оптимизации
- **Trial** — одно испытание
- **suggest_int / suggest_float / suggest_categorical** — пространство поиска

Преимущества Optuna

- **В 10 раз быстрее** GridSearch
- **Работает с условными параметрами** (например, если kernel= ‘rbf’ , то задать gamma)
- **Визуализация** процесса оптимизации
- **Pruning** — остановка бесперспективных испытаний
- **Интеграция** со sklearn, XGBoost, LightGBM

ЧАСТЬ 7. ИНТЕРПРЕТАЦИЯ ЧЕРЕЗ SHAP

7.1. Проблема «чёрного ящика»

Ансамбли — мощные, но непрозрачные. Регулятор спрашивает: **«Почему вы отклонили кредит?»**. Ответ «потому что так решила модель XGBoost» — не подходит.

7.2. Что такое SHAP?

SHAP (SHapley Additive exPlanations) — метод интерпретации, основанный на **теории кооперативных игр**.

Идея: вклад каждого признака

Представьте, что предсказание — это «приз», который нужно разделить между «игроками» (признаками) по их вкладу. SHAP считает **справедливый вклад** каждого признака через значения Шепли.

Значение Шепли (формула)

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} [v(S \cup \{i\}) - v(S)]$$

где:

- N — множество всех признаков
- S — подмножество признаков
- $v(S)$ — ценность коалиции

7.3. Свойства SHAP

Свойство	Что означает
Локальная точность	Сумма SHAP = предсказание − базовое значение
Отсутствие	Если признак не влияет, SHAP = 0
Симметрия	Одинаковые признаки = одинаковые SHAP
Согласованность	Если признак стал важнее, SHAP не уменьшится

7.4. Типы графиков SHAP

График	Что показывает
Summary plot	Глобальная важность + направление
Force plot	Локальное объяснение (одно предсказание)
Waterfall plot	Детальное объяснение одного объекта
Dependence plot	Зависимость SHAP от значения признака
Interaction plot	Влияние пары признаков

7.5. Как читать Summary plot

- **Ось Y:** признаки по важности
- **Ось X:** SHAP-значения (влияние)
- **Цвет:** значение признака (красный = высокое, синий = низкое)
- **Каждая точка:** один объект

Интерпретация:

- Красные точки справа → высокие значения **увеличивают** предсказание
- Синие точки слева → низкие значения **уменьшают** предсказание

7.6. Где применяется SHAP

Область	Задача
Банки	Объяснить отказ в кредите (требование регулятора)
Медицина	Понять, какие симптомы повлияли на диагноз
Страхование	Объяснить размер премии

HR	Объяснить, почему кандидат отклонён
Наука	Найти ключевые факторы явления

ЧАСТЬ 8. МАТРИЦА СТОИМОСТИ (GERMAN CREDIT)

8.1. Что это?

В датасете German Credit Data используется **матрица стоимости**, которая определяет «цену» ошибок:

	Предсказано: Good	**Предсказано: Bad**
Реально: Good	0	1
Реально: Bad	**5**	0

Интерпретация

- **Ошибка 1 (FP):** «Хороший клиент признан плохим» — банк теряет прибыль (1 единица)
- **Ошибка 2 (FN):** «Плохой клиент признан хорошим» — банк теряет деньги (5 единиц)

8.2. Как учесть в модели?

1. **Через `class_weight`:** Увеличить вес класса Bad в 5 раз
2. **Через порог:** Сместить порог, чтобы лучше ловить Bad
3. **Через кастомную метрику:** Оптимизировать не accuracy, а **стоимость**

8.3. Оптимизация порога

Стандартный порог 0.5 не оптимален. Перебирая пороги от 0.1 до 0.9, находим тот, при котором **суммарная стоимость минимальна**.

Это критично для бизнеса: один правильный порог может сэкономить миллионы рублей.

ЧАСТЬ 9. КЛЮЧЕВЫЕ ВЫВОДЫ

9.1. Что запомнить

1. **Ансамбль > одиночная модель.** «Мудрость толпы» работает.
2. **Bagging снижает variance,** boosting снижает bias.
3. **Random Forest** — простой, устойчивый, для baseline.
4. **XGBoost** — универсальный, точный, для соревнований.
5. **LightGBM** — быстрый, для больших данных.
6. **CatBoost** — лучший для категориальных признаков.
7. **Optuna** — современный стандарт тюнинга.
8. **SHAP** — мощный инструмент интерпретации.
9. **Матрица стоимости** — учёт бизнес-цены ошибок.

9.2. Практические рекомендации

Ситуация	Рекомендация
Первый проект	Начать с Random Forest

Много признаков, средние данные	XGBoost
Большие данные (> 1 млн)	LightGBM
Много категорий	CatBoost
Нужна интерпретация	SHAP + XGBoost
SOTA результат	Stacking из 3-5 моделей
Дисбаланс классов	`class_weight='balanced'`
Стоимость ошибок разная	Оптимизация порога

9.3. Что дальше изучать

- **Stacking и blending** — комбинирование разнородных моделей
- **Hyperopt** — альтернатива Optuna
- **AutoML** — автоматический подбор моделей и параметров
- **Feature engineering** для бустингов
- **Temporal ensembles** для временных рядов

ПРИЛОЖЕНИЕ. СЛОВАРЬ ТЕРМИНОВ

Термин	Определение
Ансамбль	Набор моделей, объединённых для улучшения качества
Base learner	Базовая (слабая) модель в ансамбле
Meta-learner	Модель, комбинирующая базовые (в stacking)

Bagging	Bootstrap Aggregating — параллельное обучение
Boosting	Последовательное обучение с исправлением ошибок
Stacking	Комбинирование разнородных моделей через мета-модель
Bootstrap	Случайная выборка с возвращением
OOB	Out-of-Bag — объекты, не попавшие в bootstrap
Bagging	Bootstrap Aggregating
Bias	Систематическая ошибка модели
Variance	Разброс предсказаний модели
Bias-Variance Tradeoff	Компромисс между смещением и дисперсией
Гиперпараметр	Параметр модели, задаваемый до обучения
Optuna	Библиотека для оптимизации гиперпараметров
SHAP	Метод интерпретации на основе теории игр
Значение Шепли	Справедливый вклад игрока в общий результат
Матрица стоимости	Таблица «цены» ошибок
Cost-sensitive learning	Обучение с учётом стоимости ошибок

ПРИЛОЖЕНИЕ. ЛИТЕРАТУРА

1. **Брейман, Л.** (2001). Random Forests. *Machine Learning*, 45(1), 5-32.

2. **Чен, Т., Гестрин, К.** (2016). XGBoost: A Scalable Tree Boosting System. *KDD '16*.
3. **Ке, Г. и др.** (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *NeurIPS 2017*.
4. **Прохоренко, А. и др.** (2018). CatBoost: unbiased boosting with categorical features. *NeurIPS 2018*.
5. **Лундберг, С., Ли, С.-И.** (2017). A Unified Approach to Interpreting Model Predictions. *NeurIPS 2017*.
6. **Акиба, Т. и др.** (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *KDD '19*.
7. **Герон, А.** (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O' Reilly.
8. **Хасти, Т., Тибширани, Р., Фридман, Дж.** (2009). *The Elements of Statistical Learning*. Springer.

📊 Ансамблевые методы — Разбор датасета German Credit Data

Часть 1. Полное описание структуры и архитектуры датасета

1.1. Общая информация о датасете

German Credit Data (Statlog) — это классический набор данных из UCI Machine Learning Repository, созданный **профессором Хансом Хофманном** в 1994 году. Датасет содержит информацию о **1000 заёмщиках** регионального банка на юге Германии за период с **1973 по 1975 год**.

Ключевые характеристики:

Параметр	Значение
Всего записей	1000
Признаков	20 + 1 целевая переменная
Классов	2 (Good / Bad)
Хороших кредитов	700 (70%)
Плохих кредитов	300 (30%)
Тип задачи	Бинарная классификация
Источник	UCI ML Repository

Особенность датасета: используется **матрица стоимости** (cost matrix), где ош ибка «плохой клиент признан хорошим» стоит в **5 раз дороже**, чем обратная ошибка.

1.2. Архитектура данных: полное описание всех 21 столбца

№	Столбец	Тип	Описание	Возможные значения
1	`checking_status`	Категориальный	Статус текущего счёта	A11: < 0 DM; A12: 0–200 DM; A13: ≥ 200 DM; A14: нет счёта
2	`duration`	Числовой	Длительность кредита (мес.)	4–72
3	`credit_history`	Категориальный	Кредитная история	A30: без кредитов; A31: погашены; A32: погашены вовремя; A33: просрочки; A34: критическая
4	`purpose`	Категориальный	Цель кредита	A40: машина; A41: мебель; A42: телевизор; A43: бытовая техника; A44: ремонт; A45: образование; A46: бизнес; A48: перекредитование; A49: другое
5	`credit_amount`	Числовой	Сумма кредита (DM)	250–18424
6	`savings_status`	Категориальный	Сбережения	A61: < 100 DM; A62: 100–500; A63: 500–

№	Столбец	Тип	Описание	Возможные значения
				1000; A64: ≥ 1000; A65: нет
7	`employment`	Категориальный	Трудовой стаж	A71: безработный; A72: < 1 года; A73: 1–4 года; A74: 4–7 лет; A75: ≥ 7 лет
8	`installment_rate`	Числовой	Доля дохода на платёж	1–4
9	`personal_status`	Категориальный	Пол и семейное положение	A91: male divorced; A92: female divorced/married; A93: male single; A94: male married
10	`other_debtors`	Категориальный	Другие поручители	A101: нет; A102: со-заявитель; A103: гарант
11	`residence_since`	Числовой	Лет в текущем жилье	1–4
12	`property`	Категориальный	Имущество	A121: недвижимость; A122: страхование; A123: машина; A124: другое
13	`age`	Числовой	Возраст (лет)	19–75

№	Столбец	Тип	Описание	Возможные значения
14	`other_installment_plans`	Категориальный	Другие рассрочки	A141: банк; A142: магазины; A143: нет
15	`housing`	Категориальный	Жильё	A151: аренда; A152: своё; A153: бесплатно
16	`existing_credits`	Числовой	Число кредитов в банке	1–4
17	`job`	Категориальный	Профессия	A171: без квалификации; A172: низкоквалиф.; A173: квалифицированный; A174: управляющий
18	`num_dependents`	Числовой	Иждивенцы	1–2
19	`telephone`	Категориальный	Телефон	A191: нет; A192: да
20	`foreign_worker`	Категориальный	Иностранный работник	A201: да; A202: нет
21	`target`	**Целевая**	Кредитоспособность	1 = Good, 2 = Bad

1.3. Статистика по категориям

Распределение целевой переменной:

- Good (кредитоспособен): 700 (70%)
- Bad (некредитоспособен): 300 (30%)

Дисбаланс классов: 70/30 — умеренный. Модель, всегда предсказывающая «Good», даст accuracy = 70%, но это обманчивый результат.

Матрица стоимости:

	Предсказано: Good	**Предсказано: Bad**
Реально: Good	0	1
Реально: Bad	**5**	0

1.4. Ключевые признаки по важности

Признак	**Важность**	**Влияние**
`checking_status`	★★★★★	Основной предиктор
`duration`	★★★★★	Длительные кредиты рискованнее
`credit_history`	★★★★★	История просрочек
`credit_amount`	★★★★	Большие суммы рискованнее
`age`	★★★	Молодые клиенты рискованнее
`purpose`	★★★	Цель кредита
`savings_status`	★★★	Отсутствие сбережений

Признак	Важность	Влияние
`employment`	★★	Стаж работы
`installment_rate`	★★	Доля дохода
`housing`	★★	Тип жилья

Часть 2. Полный разбор кода с объяснением каждой строки

Ячейка 1: Импорт библиотек

```
# ============================================================
# ЯЧЕЙКА 1: ИМПОРТ БИБЛИОТЕК
# ============================================================

import numpy as np
# NumPy — фундамент для численных операций.
# Почему: массивы NumPy быстрее списков Python в 10-100 раз.
# Где используется: математические операции, работа с массивами.

import pandas as pd
# Pandas — табличные данные (DataFrame, Series).
# Почему: удобная загрузка CSV, фильтрация, группировка, обработка пропусков.
# Где используется: загрузка German Credit Data, EDA.

import matplotlib.pyplot as plt
# Matplotlib — низкоуровневая библиотека для графиков.
# Почему: полный контроль над отрисовкой (оси, заголовки, легенды).
# Где используется: визуализация важности признаков, ROC-кривых.

import seaborn as sns
```

```
# Seaborn — статистическая визуализация поверх Matplotlib.
# Почему: красивые графики «из коробки» — heatmap, boxplot, histplot.
# Где используется: корреляционная матрица, распределения.

from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
# train_test_split — разделение данных на train/test.
# cross_val_score — автоматическая кросс-валидация.
# StratifiedKFold — сохраняет пропорции классов в фолдах.
# Почему: честная оценка модели на данных, которые она «не видела».

from sklearn.preprocessing import LabelEncoder, StandardScaler
# LabelEncoder — кодирует категории в числа (male=0, female=1).
# StandardScaler — масштабирует признаки к mean=0, std=1.
# Почему: ML-алгоритмы работают с числами, масштаб влияет на линейные модели.

from sklearn.ensemble import RandomForestClassifier
# Random Forest — ансамбль решающих деревьев (bagging).
# Почему: устойчив к переобучению, не требует масштабирования.

from xgboost import XGBClassifier
# XGBoost — градиентный бустинг от Tianqi Chen.
# Почему: SOTA-качество, регуляризация, обработка пропусков.

from lightgbm import LGBMClassifier
# LightGBM — градиентный бустинг от Microsoft.
# Почему: быстрее XGBoost в 10-20 раз, leaf-wise рост.

from catboost import CatBoostClassifier
# CatBoost — градиентный бустинг от Яндекса.
# Почему: автоматическая обработка категорий, не требует One-Hot.

from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, classification_report,
                             confusion_matrix, roc_curve)
# Метрики классификации:
# accuracy_score — доля правильных ответов.
# precision_score — точность (TP / (TP + FP)).
# recall_score — полнота (TP / (TP + FN)).
# f1_score — гармоническое среднее Precision и Recall.
```

```
# roc_auc_score — площадь под ROC-кривой.
# classification_report — полный отчёт по метрикам.
# confusion_matrix — матрица ошибок.
# roc_curve — координаты ROC-кривой.

import optuna
# Optuna — библиотека для байесовской оптимизации гиперпараметров.
# Почему: эффективнее GridSearchCV, работает с условными параметрами.

import shap
# SHAP — интерпретация моделей на основе теории игр.
# Почему: объясняет предсказания «чёрных ящиков».

import time
# time — замер времени обучения моделей.
# Почему: важно сравнивать скорость разных алгоритмов.

import warnings
warnings.filterwarnings('ignore')
# Скрываем предупреждения для чистоты вывода.
# В продакшене так делать нельзя — предупреждения надо исправлять.

print("☑ Все библиотеки успешно импортированы!")
```

Ячейка 2: Загрузка данных

```
# ============================================================
# ЯЧЕЙКА 2: ЗАГРУЗКА ДАННЫХ GERMAN CREDIT
# ============================================================

# --- Загрузка из UCI Repository ---
url = "https://archive.ics.uci.edu/ml/machine-learning-
databases/statlog/german/german.data"
# URL к оригинальному файлу german.data из UCI Repository.
# Почему именно этот URL: официальный источник, всегда доступен.

# --- Задание имён столбцов ---
columns = ['checking_status', 'duration', 'credit_history', 'purpose',
           'credit_amount', 'savings_status', 'employment', 'installment_rate',
```

```
            'personal_status', 'other_debtors', 'residence_since', 'property',
            'age', 'other_installment_plans', 'housing', 'existing_credits',
            'job', 'num_dependents', 'telephone', 'foreign_worker', 'target']
# Список из 21 имени столбца.
# Почему именно эти имена: соответствуют описанию UCI.
# Где взять: https://archive.ics.uci.edu/ml/datasets/statlog+(german+credit+data)

# --- Чтение CSV ---
df = pd.read_csv(url, sep=' ', header=None, names=columns)
# pd.read_csv() — основная функция чтения.
# sep=' ' — разделитель: пробел (в исходном файле пробелы).
# header=None — в файле НЕТ строки заголовков, задаём вручную.
# names=columns — присваиваем имена столбцам.
# Почему не sep=',', как по умолчанию: исходный файл использует пробелы.

# --- Целевая переменная ---
df['target'] = (df['target'] == 2).astype(int)
# В исходных данных: 1 = Good, 2 = Bad.
# Преобразование: (target == 2) → True для Bad, False для Good.
# .astype(int) → True=1, False=0.
# Итог: 0 = Good, 1 = Bad (удобно для sklearn: 1 — положительный класс).

# --- Первичный осмотр ---
print("=" * 60)
print("РАЗМЕР ДАТАСЕТА:")
print("=" * 60)
print(f"Строк: {df.shape[0]}, Столбцов: {df.shape[1]}")
# .shape — кортеж (строки, столбцы).
# Ожидаем: 1000 строк, 21 столбец.

print(f"\nРаспределение целевой:")
print(df['target'].value_counts())
# .value_counts() — частота уникальных значений.
# Ожидаем: 0 → 700, 1 → 300.

print(f"\nТипы данных:")
print(df.dtypes.value_counts())
# .dtypes — типы столбцов.
# .value_counts() — сколько столбцов каждого типа.
```

```
# Ожидаем: object (14), int64 (7).
```

Ячейка 3: EDA — анализ пропусков и целевой переменной

```
# ============================================================
# ЯЧЕЙКА 3: EDA — АНАЛИЗ ПРОПУСКОВ И ЦЕЛЕВОЙ
# ============================================================

# --- Проверка пропусков ---
missing = df.isnull().sum()
# .isnull() — маска True/False (True где NaN).
# .sum() — количество True по столбцам.

print("=" * 60)
print("ПРОПУСКИ В ДАННЫХ:")
print("=" * 60)
print(f"Всего пропусков: {missing.sum()}")
# .sum().sum() — двойная сумма: по столбцам, потом по всему DataFrame.
# German Credit Data НЕ содержит пропусков.

# --- Визуализация целевой переменной ---
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
# 2 графика рядом.

# График 1: Countplot
sns.countplot(data=df, x='target', ax=axes[0], palette=['#2ecc71', '#e74c3c'])
# sns.countplot — столбчатая диаграмма частот.
# x='target' — ось X, palette — цвета.
axes[0].set_title('Распределение целевой переменной', fontsize=14)
axes[0].set_xticklabels(['Good (0)', 'Bad (1)'])
axes[0].set_ylabel('Количество заёмщиков')

# График 2: Pie chart
df['target'].value_counts().plot.pie(autopct='%1.1f%%', ax=axes[1],
                                     colors=['#2ecc71', '#e74c3c'],
                                     labels=['Good', 'Bad'])
# .plot.pie() — круговая диаграмма.
# autopct='%1.1f%%' — формат подписи (1 знак после запятой, проценты).
```

```
axes[1].set_title('Доля классов', fontsize=14)
axes[1].set_ylabel('')

plt.tight_layout()
plt.show()

# --- Вывод ---
print("\n" + "=" * 60)
print("ВЫВОД:")
print("=" * 60)
print("Дисбаланс классов: 70% Good / 30% Bad")
print("➡️ Модель, всегда предсказывающая Good, даст accuracy = 70%")
print("➡️ Нужно использовать Precision, Recall, F1, ROC-AUC")
print("➡️ Учесть матрицу стоимости (ошибка FN в 5 раз дороже FP)")
```

Ячейка 4: EDA — корреляции и распределения

```
# ============================================================
# ЯЧЕЙКА 4: EDA — КОРРЕЛЯЦИИ И РАСПРЕДЕЛЕНИЯ
# ============================================================

# --- Числовые признаки ---
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
# .select_dtypes(include=[np.number]) — выбираем только числовые.
# .columns.tolist() — преобразуем в список.

print("Числовые признаки:", numeric_cols)

# --- Корреляционная матрица ---
plt.figure(figsize=(14, 10))
corr_matrix = df[numeric_cols].corr()
# .corr() — матрица корреляций Пирсона.

sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, square=True, linewidths=0.5)
# sns.heatmap — тепловая карта.
# annot=True — показывать числа.
# fmt='.2f' — 2 знака после запятой.
```

```
# center=0 — белый цвет = 0.

plt.title('Корреляционная матрица числовых признаков', fontsize=14)
plt.tight_layout()
plt.show()

# --- Boxplot: age vs target ---
plt.figure(figsize=(10, 5))
sns.boxplot(data=df, x='target', y='age', palette=['#2ecc71', '#e74c3c'])
# Boxplot показывает медиану, квартили, выбросы.
plt.title('Возраст по кредитоспособности', fontsize=14)
plt.xticks([0, 1], ['Good', 'Bad'])
plt.show()

# --- Boxplot: duration vs target ---
plt.figure(figsize=(10, 5))
sns.boxplot(data=df, x='target', y='duration', palette=['#2ecc71', '#e74c3c'])
plt.title('Длительность кредита по кредитоспособности', fontsize=14)
plt.xticks([0, 1], ['Good', 'Bad'])
plt.show()
```

Ячейка 5: Предобработка — кодирование категорий

```
# ============================================================
# ЯЧЕЙКА 5: ПРЕДОБРАБОТКА — КОДИРОВАНИЕ КАТЕГОРИЙ
# ============================================================

# --- Определяем категориальные столбцы ---
cat_cols = df.select_dtypes(include=['object']).columns.tolist()
# .select_dtypes(include=['object']) — выбираем object-столбцы.

print(f"Категориальных признаков: {len(cat_cols)}")
print("Список:", cat_cols)

# --- Label Encoding ---
le_dict = {}
# Словарь для хранения кодировщиков.
# Почему: нужно сохранить для будущего инференса.
```

```
for col in cat_cols:
    le = LabelEncoder()
    # Создаём кодировщик для каждого столбца.

    df[col] = le.fit_transform(df[col])
    # .fit_transform() — обучает кодировщик и сразу преобразует.
    # Каждой категории присваивается число 0..n-1.

    le_dict[col] = le
    # Сохраняем кодировщик в словарь.

print(f"\n☑ Закодировано {len(cat_cols)} категориальных признаков")

# --- Проверка ---
print("\nПервые 5 строк после кодирования:")
print(df.head())

# --- Разделение на X и y ---
X = df.drop('target', axis=1)
# X — матрица признаков (все столбцы, кроме target).
# axis=1 — удаляем столбец (axis=0 — строку).

y = df['target']
# y — вектор целевой переменной.

print(f"\nРазмер X: {X.shape}")
print(f"Размер y: {y.shape}")

# --- Разделение на train/test ---
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,           # 20% в test
    random_state=42,         # воспроизводимость
    stratify=y               # сохраняет пропорцию классов
)
# stratify=y — КРИТИЧНО для дисбаланса:
# без стратификации в test могло оказаться 50% Bad.
```

```
print(f"\nTrain: {X_train.shape}, Test: {X_test.shape}")
print(f"Доля Bad в train: {y_train.mean():.4f}")
print(f"Доля Bad в test:  {y_test.mean():.4f}")
# Пропорции должны совпадать благодаря stratify.
```

Ячейка 6: Random Forest — обучение и оценка

```
# ============================================================
# ЯЧЕЙКА 6: RANDOM FOREST — ОБУЧЕНИЕ И ОЦЕНКА
# ============================================================

# --- Создание модели ---
rf_model = RandomForestClassifier(
    n_estimators=200,        # 200 деревьев (default=100)
    max_depth=10,            # ограничиваем глубину (default=None)
    min_samples_split=5,     # минимум 5 образцов для разбиения (default=2)
    min_samples_leaf=2,      # минимум 2 образца в листе (default=1)
    max_features='sqrt',     # √p признаков на разбиение (default='sqrt')
    class_weight='balanced', # учитываем дисбаланс классов
    random_state=42,         # воспроизводимость
    n_jobs=-1                # все ядра CPU
)
# Почему такие параметры:
# n_estimators=200 — больше деревьев = стабильнее (но медленнее).
# max_depth=10 — ограничиваем, чтобы не переобучаться.
# class_weight='balanced' — компенсируем дисбаланс 70/30.

# --- Обучение ---
start = time.time()
rf_model.fit(X_train, y_train)
rf_time = time.time() - start
# Замер времени обучения.

# --- Предсказания ---
y_pred_rf = rf_model.predict(X_test)
y_proba_rf = rf_model.predict_proba(X_test)[:, 1]
# predict_proba — вероятности [P(0), P(1)].
# [:, 1] — берём вероятность класса 1 (Bad).
```

```
# --- Метрики ---
print("=" * 60)
print("RANDOM FOREST:")
print("=" * 60)
print(f"⏱️  Время обучения: {rf_time:.3f} сек")
print(f"Accuracy:  {accuracy_score(y_test, y_pred_rf):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_rf):.4f}")
print(f"Recall:    {recall_score(y_test, y_pred_rf):.4f}")
print(f"F1-score:  {f1_score(y_test, y_pred_rf):.4f}")
print(f"ROC-AUC:   {roc_auc_score(y_test, y_proba_rf):.4f}")

# --- Кросс-валидация ---
cv_scores = cross_val_score(
    rf_model, X_train, y_train,
    cv=StratifiedKFold(5, shuffle=True, random_state=42),
    scoring='roc_auc'
)
print(f"\nCV ROC-AUC: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}")

# --- Важность признаков ---
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf_model.feature_importances_
}).sort_values('Importance', ascending=False)

print("\nТОП-10 важных признаков:")
print(feature_importance.head(10).to_string(index=False))
```

Ячейка 7: XGBoost, LightGBM, CatBoost — сравнение

```
# ============================================================
# ЯЧЕЙКА 7: СРАВНЕНИЕ XGBOOST, LIGHTGBM, CATBOOST
# ============================================================

def evaluate_model(model, X_train, X_test, y_train, y_test, name):
    """Обучает модель, замеряет время, считает метрики."""
    start = time.time()
```

```
    model.fit(X_train, y_train)
    train_time = time.time() - start

    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    return {
        'Model': name,
        'Time (sec)': round(train_time, 3),
        'Accuracy': round(accuracy_score(y_test, y_pred), 4),
        'Precision': round(precision_score(y_test, y_pred), 4),
        'Recall': round(recall_score(y_test, y_pred), 4),
        'F1': round(f1_score(y_test, y_pred), 4),
        'ROC-AUC': round(roc_auc_score(y_test, y_proba), 4)
    }

# --- XGBoost ---
xgb_model = XGBClassifier(
    n_estimators=200,        # число boosting-раундов
    max_depth=6,             # глубина деревьев
    learning_rate=0.1,       # скорость обучения
    subsample=0.9,           # доля объектов для обучения
    colsample_bytree=0.9,    # доля признаков для обучения
    gamma=0.1,               # мин. снижение потерь для разбиения
    reg_alpha=0.1,           # L1-регуляризация
    reg_lambda=1.0,          # L2-регуляризация
    random_state=42,
    eval_metric='logloss',
    use_label_encoder=False
)
results_xgb = evaluate_model(xgb_model, X_train, X_test, y_train, y_test, 'XGBoost')

# --- LightGBM ---
lgbm_model = LGBMClassifier(
    n_estimators=200,
    num_leaves=31,           # число листьев (важнее max_depth)
    max_depth=7,
    learning_rate=0.05,
    subsample=0.8,
```

```
    colsample_bytree=0.8,
    min_child_samples=20,
    random_state=42,
    verbose=-1
)
results_lgbm = evaluate_model(lgbm_model, X_train, X_test, y_train, y_test,
'LightGBM')


# --- CatBoost ---
cat_model = CatBoostClassifier(
    iterations=200,          # число итераций (аналог n_estimators)
    depth=6,
    learning_rate=0.1,
    l2_leaf_reg=3,           # L2-регуляризация
    random_state=42,
    verbose=0                # без подробного вывода
)
results_cat = evaluate_model(cat_model, X_train, X_test, y_train, y_test, 'CatBoost')


# --- Сводная таблица ---
print("=" * 90)
print("СРАВНЕНИЕ ГРАДИЕНТНЫХ БУСТИНГОВ:")
print("=" * 90)
results_df = pd.DataFrame([results_xgb, results_lgbm, results_cat])
print(results_df.to_string(index=False))
```

Ячейка 8: Optuna — оптимизация LightGBM

```
# ============================================================
# ЯЧЕЙКА 8: ОПТИМИЗАЦИЯ LIGHTGBM С ПОМОЩЬЮ OPTUNA
# ============================================================

optuna.logging.set_verbosity(optuna.logging.WARNING)
# Скрываем подробный вывод Optuna.

def objective(trial):
    """Objective-функция: обучает LightGBM и возвращает CV-метрику."""
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 50, 500, step=50),
```

```
        # suggest_int — целое число от 50 до 500 с шагом 50.

        'num_leaves': trial.suggest_int('num_leaves', 10, 100, step=5),
        # Число листьев — ключевой параметр LightGBM.

        'max_depth': trial.suggest_int('max_depth', 3, 12),
        # Глубина дерева.

        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3, log=True),
        # log=True — логарифмическая шкала (для learning_rate это правильно).

        'subsample': trial.suggest_float('subsample', 0.5, 1.0),
        # Доля объектов для обучения.

        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.5, 1.0),
        # Доля признаков для обучения.

        'min_child_samples': trial.suggest_int('min_child_samples', 5, 50),
        # Минимум образцов в листе.

        'reg_alpha': trial.suggest_float('reg_alpha', 1e-3, 10, log=True),
        # L1-регуляризация.

        'reg_lambda': trial.suggest_float('reg_lambda', 1e-3, 10, log=True),
        # L2-регуляризация.
    }

    model = LGBMClassifier(**params, random_state=42, verbose=-1)
    # Создаём модель с предложенными параметрами.

    cv_scores = cross_val_score(
        model, X_train, y_train,
        cv=StratifiedKFold(5, shuffle=True, random_state=42),
        scoring='roc_auc'
    )
    # 5-фолдовая кросс-валидация с ROC-AUC.

    return cv_scores.mean()
    # Optuna будет максимизировать эту метрику.
```

```
# --- Запуск оптимизации ---
study = optuna.create_study(direction='maximize')
# direction='maximize' — хотим максимизировать ROC-AUC.

study.optimize(objective, n_trials=30, show_progress_bar=True)
# 30 испытаний — компромисс между временем и качеством.

print("=" * 60)
print("ЛУЧШИЕ ПАРАМЕТРЫ LIGHTGBM:")
print("=" * 60)
print(f"Best ROC-AUC: {study.best_value:.4f}")
for key, value in study.best_params.items():
    print(f"  {key}: {value}")

# --- Обучение с лучшими параметрами ---
best_lgbm = LGBMClassifier(**study.best_params, random_state=42, verbose=-1)
best_lgbm.fit(X_train, y_train)

y_pred_best = best_lgbm.predict(X_test)
y_proba_best = best_lgbm.predict_proba(X_test)[:, 1]

print("\n" + "=" * 60)
print("РЕЗУЛЬТАТЫ С ЛУЧШИМИ ПАРАМЕТРАМИ:")
print("=" * 60)
print(f"Accuracy:  {accuracy_score(y_test, y_pred_best):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_best):.4f}")
print(f"Recall:    {recall_score(y_test, y_pred_best):.4f}")
print(f"F1:        {f1_score(y_test, y_pred_best):.4f}")
print(f"ROC-AUC:   {roc_auc_score(y_test, y_proba_best):.4f}")
```

Ячейка 9: SHAP — интерпретация

```
# ============================================================
# ЯЧЕЙКА 9: SHAP — ИНТЕРПРЕТАЦИЯ LIGHTGBM
# ============================================================

# --- Создаём explainer ---
```

```
explainer = shap.TreeExplainer(best_lgbm)
# TreeExplainer — быстрый алгоритм для древовидных моделей.
# Почему не KernelExplainer: он медленнее в 100-1000 раз.

# --- Вычисляем SHAP-значения ---
shap_values = explainer.shap_values(X_test)
# Для бинарной классификации shap_values — список из двух массивов.
# [0] — для класса 0 (Good), [1] — для класса 1 (Bad).

if isinstance(shap_values, list):
    shap_values_class1 = shap_values[1]
else:
    shap_values_class1 = shap_values
# Обработка случая, когда shap_values — уже массив.

print(f"Форма SHAP-значений: {shap_values_class1.shape}")
# (количество объектов, количество признаков).

# --- Summary plot ---
print("\n" + "=" * 60)
print("SUMMARY PLOT — глобальная важность:")
print("=" * 60)
shap.summary_plot(shap_values_class1, X_test, plot_type="dot")
# Каждая точка — один объект.
# Ось X — SHAP-значение (влияние на предсказание).
# Цвет — значение признака (красный = высокое, синий = низкое).

# --- Bar plot (важность) ---
shap.summary_plot(shap_values_class1, X_test, plot_type="bar")
# Столбчатая диаграмма средней абсолютной важности.

# --- Waterfall plot для одного объекта ---
worst_idx = np.argmax(y_proba_best)
# Индекс объекта с наибольшей вероятностью "Bad".

print(f"\nОбъект #{worst_idx}, вероятность Bad = {y_proba_best[worst_idx]:.4f}")

shap.plots.waterfall(
    shap.Explanation(
```

```
        values=shap_values_class1[worst_idx],
        base_values=explainer.expected_value[1] if
isinstance(explainer.expected_value, list) else explainer.expected_value,
        data=X_test.iloc[worst_idx],
        feature_names=X_test.columns.tolist()
    )
)
# Waterfall показывает вклад каждого признака в конкретное предсказание.

# --- Dependence plot ---
top_feature = feature_importance.iloc[0]['Feature']
print(f"\nТоп-1 признак: {top_feature}")

shap.dependence_plot(
    top_feature,
    shap_values_class1,
    X_test,
    interaction_index=None
)
# Dependence plot показывает, как SHAP зависит от значения признака.
```

Ячейка 10: Оптимизация порога с учётом стоимости

```
# ============================================================
# ЯЧЕЙКА 10: ОПТИМИЗАЦИЯ ПОРОГА С УЧЁТОМ МАТРИЦЫ СТОИМОСТИ
# ============================================================

def cost_metric(y_true, y_pred):
    """Считает общую стоимость ошибок по матрице стоимости."""
    cost_matrix = np.array([[0, 1], [5, 0]])
    # Строки — истина, столбцы — предсказание.
    # [0,0] = 0 (TN), [0,1] = 1 (FP)
    # [1,0] = 5 (FN), [1,1] = 0 (TP)

    total_cost = 0
    for true, pred in zip(y_true, y_pred):
        total_cost += cost_matrix[true, pred]
    return total_cost
```

```
# --- Подбор порога ---
thresholds = np.arange(0.05, 0.95, 0.05)
costs = []

for thresh in thresholds:
    y_pred_thresh = (y_proba_best >= thresh).astype(int)
    # (prob >= thresh) — True/False.
    # .astype(int) — True=1, False=0.

    cost = cost_metric(y_test.values, y_pred_thresh)
    costs.append(cost)

# --- Результаты ---
best_idx = np.argmin(costs)
# np.argmin — индекс минимального значения.

best_threshold = thresholds[best_idx]

print("=" * 70)
print("ОПТИМИЗАЦИЯ ПОРОГА:")
print("=" * 70)
print(f"Стандартный порог 0.5: стоимость = {cost_metric(y_test.values, (y_proba_best
>= 0.5).astype(int))}")
print(f"Оптимальный порог:    {best_threshold:.2f}")
print(f"Минимальная стоимость: {min(costs)}")

# --- Визуализация ---
plt.figure(figsize=(12, 5))
plt.plot(thresholds, costs, color='#00d4ff', linewidth=2.5, marker='o')
plt.axvline(x=best_threshold, color='#ff006e', linestyle='--', label=f'Оптимум =
{best_threshold:.2f}')
plt.xlabel('Порог классификации', fontsize=12)
plt.ylabel('Стоимость ошибок', fontsize=12)
plt.title('Зависимость стоимости ошибок от порога', fontsize=14)
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# --- Итоговые метрики с оптимальным порогом ---
```

```
y_pred_optimal = (y_proba_best >= best_threshold).astype(int)

print("\n" + "=" * 70)
print("МЕТРИКИ С ОПТИМАЛЬНЫМ ПОРОГОМ:")
print("=" * 70)
print(f"Accuracy:  {accuracy_score(y_test, y_pred_optimal):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_optimal):.4f}")
print(f"Recall:    {recall_score(y_test, y_pred_optimal):.4f}")
print(f"F1:        {f1_score(y_test, y_pred_optimal):.4f}")
print(f"Стоимость: {cost_metric(y_test.values, y_pred_optimal)}")
```

Ячейка 11: Итоговое сравнение всех моделей

```
# ============================================================
# ЯЧЕЙКА 11: ИТОГОВОЕ СРАВНЕНИЕ ВСЕХ МОДЕЛЕЙ
# ============================================================

# --- Собираем результаты ---
all_results = pd.DataFrame([
    {'Model': 'Random Forest', **{k: v for k, v in results_rf.items() if k != 
'Model'}},
    # **{...} — распаковка словаря.
])
# ... (аналогично для других моделей)

# --- Сводная таблица ---
print("=" * 110)
print("ИТОГОВОЕ СРАВНЕНИЕ ВСЕХ МОДЕЛЕЙ:")
print("=" * 110)

final_comparison = pd.DataFrame([
    results_rf,
    results_xgb,
    results_lgbm,
    results_cat,
    {'Model': 'LightGBM (Optuna)', **results_best}
])
print(final_comparison.to_string(index=False))
```

```
# --- Выводы ---
print("\n" + "=" * 60)
print("КЛЮЧЕВЫЕ ВЫВОДЫ:")
print("=" * 60)
print("""
1. Все ансамбли превосходят одиночные модели.
2. LightGBM — самый быстрый.
3. CatBoost — хорош для категорий.
4. Optuna даёт +2-5% к ROC-AUC.
5. Матрица стоимости критична для бизнеса.
6. SHAP объясняет предсказания.
""")
```

📋 Итоговый список операторов

Pandas

Оператор	Назначение
`pd.read_csv()`	Чтение CSV
`df.shape`	(строки, столбцы)
`df.dtypes`	Типы данных
`df.isnull().sum()`	Пропуски
`df['col'].value_counts()`	Частоты
`df.select_dtypes()`	Выбор по типу

Оператор	Назначение
`df.corr()`	Корреляции
`df.drop()`	Удаление столбца
`df.head()`	Первые 5 строк
`pd.DataFrame()`	Создание DataFrame
`df.sort_values()`	Сортировка

Scikit-learn

Оператор	Назначение
`train_test_split()`	Разделение
`LabelEncoder()`	Кодирование категорий
`RandomForestClassifier()`	Random Forest
`cross_val_score()`	Кросс-валидация
`StratifiedKFold()`	Стратифицированные фолды
`accuracy_score()`	Доля верных
`precision_score()`	Precision

Оператор	Назначение
`recall_score()`	Recall
`f1_score()`	F1
`roc_auc_score()`	ROC-AUC

XGBoost / LightGBM / CatBoost

Оператор	Назначение
`XGBClassifier()`	XGBoost
`LGBMClassifier()`	LightGBM
`CatBoostClassifier()`	CatBoost

Optuna / SHAP

Оператор	Назначение
`optuna.create_study()`	Создание исследования
`study.optimize()`	Запуск оптимизации
`trial.suggest_int()`	Целое число

Оператор	Назначение
`trial.suggest_float()`	Дробное число
`shap.TreeExplainer()`	Explainer для деревьев
`shap.summary_plot()`	Глобальная важность
`shap.waterfall()`	Локальное объяснение

🎯 Вывод

Разбор кода показывает **полный цикл ML-проекта** для задачи кредитного ск оринга:

1. **Загрузка данных** из UCI Repository
2. **EDA** — анализ пропусков, распределений, корреляций
3. **Предобработка** — Label Encoding, стратифицированное разделение
4. **Обучение 4 моделей** — Random Forest, XGBoost, LightGBM, CatBoost
5. **Оптимизация гиперпараметров** через Optuna
6. **Интерпретация** через SHAP
7. **Оптимизация порога** с учётом матрицы стоимости
8. **Сравнение результатов** всех моделей

Ожидаемые метрики: ROC-AUC ~0.78–0.82, с учётом стоимости ошибок мож но сэкономить до 30% потерь банка.